# GraphQL v/s REST

**What is GraphQL?**

**GraphQL is a query language for APIs and a runtime for executing those queries.**

**Core Idea (In One Line)**

**The client asks for exactly the data it needs — nothing more, nothing less.**

**Problem with REST:**

**You often get:**

- **Over-fetching (too much data)**
- **Under-fetching (too little data, need multiple requests)**

**GraphQL Fixes This**

**REST (Representational State Transfer) is an architectural style where:**

- **Each resource has its own endpoint**
- **Uses HTTP methods (GET, POST, PUT, DELETE)**
- **Server defines response structure**

**GraphQL is a query language for APIs where:**

- **Client defines what data it needs**
- **Single endpoint (usually /graphql)**
- **Server resolves fields dynamically**

| Feature | REST | GraphQL |
|---|---|---|
| Endpoints | Multiple endpoints | Single endpoint |
| Data fetching | Fixed response | Client chooses fields |
| Over-fetching | Common | Avoided |
| Under-fetching | Common | Avoided |
| Versioning | Required for breaking changes | Rarely needed |
| Caching | Easy (HTTP caching) | More complex |
| Learning curve | Easier | Steeper |
| Error handling | HTTP status codes | 200 + error object |
| Real-time | Needs WebSockets separately | Built-in subscriptions |

**Performance Consideration**

**REST:**

- **Simpler**
- **Predictable**
- **Easier to cache at CDN level**

**GraphQL:**

- **Can reduce network calls**
- **But complex queries can be heavy**
- **Needs query depth limiting**

**When to Use REST**

**Use REST when:**

- **Simple CRUD application**

- **Public API**
- **Strong caching needed**
- **Team is small**
- **Mobile/web requirements are simple**

**Most SaaS apps still use REST.**

**When to Use GraphQL**

**Use GraphQL when:**

- **Complex frontend (like dashboards)**
- **Multiple clients (web, mobile, tablet)**
- **Data relationships are deep**
- **You want flexible querying**

**Companies like:**

- **Meta**
- **Shopify**
- **GitHub**

**Use GraphQL heavily.**

**Real-World Trade-Off**

**GraphQL is powerful but:**

- **Harder to cache**
- **Harder to monitor**
- **Requires schema design discipline**
- **Can cause performance issues if abused**

**REST is:**

- **Simpler**

- **More predictable**
- **Easier for most teams**

**REST exposes multiple endpoints and the server controls response structure. GraphQL uses a single endpoint where the client specifies exactly what data it needs, reducing over-fetching and under-fetching. REST is simpler and easier to cache, while GraphQL is more flexible and better suited for complex frontends. Choice depends on project requirements and team maturity.”**